


# Symbiosis Institute of Technology, Pune

## Faculty of Engineering

## CSE- Academic Year 2026-27

## Java Programming – Lab Batch 2025-29

| Lab Assignment No:- 24 | |
|---|---|
| | |
| **Name of Student** | Arpit Chakraborty |
| **PRN No.** | 25070122047 |
| **Batch** | 2025-29 |
| **Class** | CSE A2 |
| **Academic Year & Semester** | 2026-27, III |
| **Date of Submission** | 27/10/26 |
| | |
| **Title of Assignment:** | Develop a Java GUI application integrated with JDBC to perform database operations. |
| **Practice questions**<br><br>**Output Screenshots Solve questions on given link and attach screen shot of leetcode with your login and o/p** | 1. Create a Library Management System GUI connected with database<br>2. Create a Book Issue Tracking System GUI application using Java Swing connected to a database. The system should store book ID, student name, issue date, and return date, and allow record management.<br>import javax.swing.*;<br>import javax.swing.table.DefaultTableModel;<br>import java.awt.*;<br>import java.sql.*;<br>import java.time.LocalDate;<br><br>public class LibraryManagementSystem extends JFrame { |

```
static final String URL =
    "jdbc:mysql://localhost:3306/librarydb";
static final String USER = "root";
static final String PASSWORD = "arpit@2006";

JTextField bookId = new JTextField(12);
JTextField studentName = new JTextField(15);
JTextField issueDate = new JTextField(12);
JTextField returnDate = new JTextField(12);

DefaultTableModel model = new DefaultTableModel(
    new String[]{"Book ID", "Student Name",
                "Issue Date", "Return Date"}, 0
);

JTable table = new JTable(model);
Connection con;

public LibraryManagementSystem() {
    setTitle("Library Management System");
    setSize(800, 500);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);

    JPanel form = new JPanel(new GridLayout(4, 2, 8, 8));
    form.setBorder(BorderFactory.createTitledBorder(
        "Book Issue Details"));

    form.add(new JLabel("Book ID:"));
    form.add(bookId);

    form.add(new JLabel("Student Name:"));
    form.add(studentName);
```

```
form.add(new JLabel("Issue Date (YYYY-MM-DD):"));
form.add(issueDate);

form.add(new JLabel("Return Date (YYYY-MM-DD):"));
form.add(returnDate);

JButton add = new JButton("Add");
JButton display = new JButton("Display");
JButton update = new JButton("Update");
JButton delete = new JButton("Delete");
JButton clear = new JButton("Clear");

JPanel buttons = new JPanel();
buttons.add(add);
buttons.add(display);
buttons.add(update);
buttons.add(delete);
buttons.add(clear);

setLayout(new BorderLayout(10, 10));
add(form, BorderLayout.NORTH);
add(new JScrollPane(table), BorderLayout.CENTER);
add(buttons, BorderLayout.SOUTH);

add.addActionListener(e -> addRecord());
display.addActionListener(e -> displayRecords());
update.addActionListener(e -> updateRecord());
delete.addActionListener(e -> deleteRecord());
clear.addActionListener(e -> clearFields());

table.getSelectionModel().addListSelectionListener(e -> {
    if (!e.getValueIsAdjusting()
            && table.getSelectedRow() >= 0) {
        int r = table.convertRowIndexToModel(
```

```
                table.getSelectedRow());

            bookId.setText(model.getValueAt(r, 0).toString());
            studentName.setText(
                model.getValueAt(r, 1).toString());
            issueDate.setText(
                model.getValueAt(r, 2).toString());
            returnDate.setText(
                model.getValueAt(r, 3).toString());

            bookId.setEditable(false);
        }
    });

    connectDatabase();
    if (con != null) {
        displayRecords();
    }
}

void connectDatabase() {
    try {
        Class.forName("com.mysql.cj.jdbc.Driver");

        con = DriverManager.getConnection(
            URL, USER, PASSWORD);

        JOptionPane.showMessageDialog(this,
            "Library database connected successfully.");

    } catch (Exception e) {
        JOptionPane.showMessageDialog(this,
            "Database connection failed: "
            + e.getMessage());
```

```
    }
}

boolean validateInput() {
    try {
        if (bookId.getText().trim().isEmpty()
                || studentName.getText().trim().isEmpty()
                || issueDate.getText().trim().isEmpty()
                || returnDate.getText().trim().isEmpty()) {
            JOptionPane.showMessageDialog(this,
                "Please fill in all fields.");
            return false;
        }

        LocalDate issue =
            LocalDate.parse(issueDate.getText().trim());
        LocalDate ret =
            LocalDate.parse(returnDate.getText().trim());

        Integer.parseInt(bookId.getText().trim());

        if (ret.isBefore(issue)) {
            JOptionPane.showMessageDialog(this,
                "Return date cannot be before issue date.");
            return false;
        }

        return true;

    } catch (Exception e) {
        JOptionPane.showMessageDialog(this,
            "Enter a valid Book ID and dates in YYYY-MM-DD format.");
        return false;
    }
```

```
}

void addRecord() {
    if (con == null || !validateInput()) return;

    String sql = "INSERT INTO book_issues "
        + "(book_id, student_name, issue_date, return_date) "
        + "VALUES (?, ?, ?, ?)";

    try (PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setInt(1, Integer.parseInt(bookId.getText().trim()));
        ps.setString(2, studentName.getText().trim());
        ps.setDate(3, Date.valueOf(issueDate.getText().trim()));
        ps.setDate(4, Date.valueOf(returnDate.getText().trim()));

        ps.executeUpdate();

        JOptionPane.showMessageDialog(this,
            "Record added successfully.");

        displayRecords();
        clearFields();

    } catch (SQLException e) {
        JOptionPane.showMessageDialog(this,
            "Insert failed: " + e.getMessage());
    }
}

void displayRecords() {
    if (con == null) return;

    model.setRowCount(0);
```

```
    String sql = "SELECT * FROM book_issues ORDER BY book_id";

    try (Statement st = con.createStatement();
         ResultSet rs = st.executeQuery(sql)) {

        while (rs.next()) {
            model.addRow(new Object[]{
                rs.getInt("book_id"),
                rs.getString("student_name"),
                rs.getDate("issue_date"),
                rs.getDate("return_date")
            });
        }

    } catch (SQLException e) {
        JOptionPane.showMessageDialog(this,
            "Display failed: " + e.getMessage());
    }
}

void updateRecord() {
    if (con == null || !validateInput()) return;

    String sql = "UPDATE book_issues SET "
        + "student_name=?, issue_date=?, return_date=? "
        + "WHERE book_id=?";

    try (PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setString(1, studentName.getText().trim());
        ps.setDate(2, Date.valueOf(issueDate.getText().trim()));
        ps.setDate(3, Date.valueOf(returnDate.getText().trim()));
        ps.setInt(4, Integer.parseInt(bookId.getText().trim()));

        int rows = ps.executeUpdate();
```

```
            JOptionPane.showMessageDialog(this,
                rows > 0 ? "Record updated successfully."
                         : "Book ID not found.");

            displayRecords();
            clearFields();

        } catch (SQLException e) {
            JOptionPane.showMessageDialog(this,
                "Update failed: " + e.getMessage());
        }
    }

    void deleteRecord() {
        if (con == null) return;

        if (bookId.getText().trim().isEmpty()) {
            JOptionPane.showMessageDialog(this,
                "Select a record or enter a Book ID.");
            return;
        }

        try (PreparedStatement ps = con.prepareStatement(
                "DELETE FROM book_issues WHERE book_id=?")) {

            ps.setInt(1, Integer.parseInt(bookId.getText().trim()));

            int rows = ps.executeUpdate();

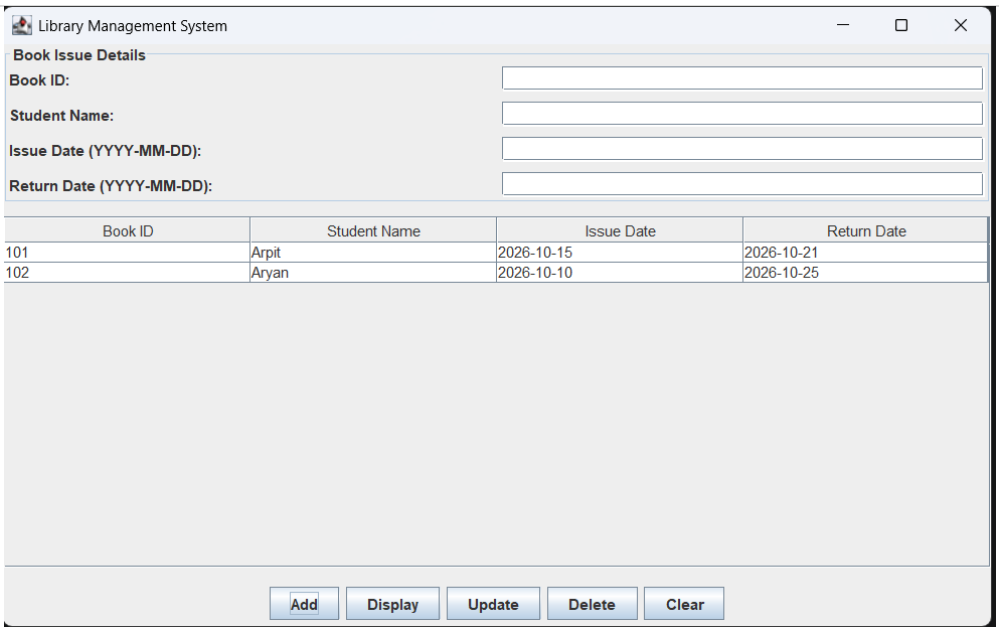
            JOptionPane.showMessageDialog(this,
                rows > 0 ? "Record deleted successfully."
                         : "Book ID not found.");
```

```
            displayRecords();
            clearFields();

        } catch (Exception e) {
            JOptionPane.showMessageDialog(this,
                "Delete failed: " + e.getMessage());
        }
    }

    void clearFields() {
        bookId.setText("");
        studentName.setText("");
        issueDate.setText("");
        returnDate.setText("");
        bookId.setEditable(true);
        table.clearSelection();
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            new LibraryManagementSystem().setVisible(true);
        });
    }
}
```

| | |
|---|---|
| **Output Screenshots (if applicable)** |  |
| **Conclusion** | Thus, a Library Management System GUI application was developed using Java Swing and JDBC connected to a MySQL database. The application allows users to add, display, update, and delete book issue records, including Book ID, student name, issue date, and return date. This practical helped in understanding GUI development, database connectivity, and record management using Java. |